Import Library

In [2]:
```python
import os
import pandas as pd
import random
import matplotlib.pyplot as plt
import keras
import tensorflow as tf
import string
import re
import numpy as np
import warnings
warnings.filterwarnings("ignore")
```

In [3]:
```python
print(f"Tensor Flow Version: {tf.__version__}")
print()
gpu = len(tf.config.list_physical_devices('GPU'))>0
print("GPU is", "AVAILABLE" if gpu else "NOT AVAILABLE")
```

```
Tensor Flow Version: 2.18.0

GPU is AVAILABLE
```

Preparing the data

In [4]:
```python
# get the file path
path = "/kaggle/input"
path = path if os.path.exists(path) else ".{}".format(path)
file_path = os.path.join(path, "translation", "eng_-french.csv")

# read the data
df = pd.read_csv(file_path)
df['source'] = df['English words/sentences']

# let's add an initial "seed" token ([start]) and a stop token ([end]) to each target sentence.
df['target'] = df['French words/sentences'].apply(lambda x: '[start] ' + x + ' [end]')
df = df.drop(['English words/sentences', 'French words/sentences'], axis=1)

# display a few random samples
df.sample(5)
```

Out[4]:

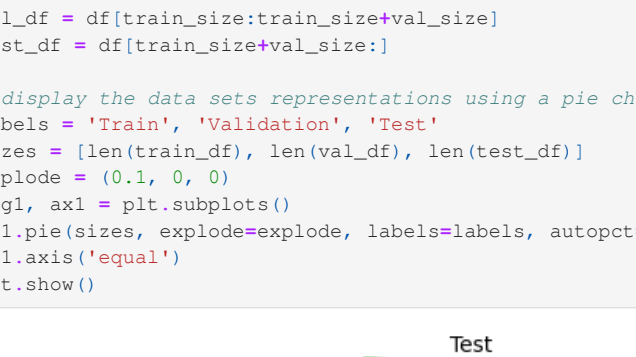
	source	target
36804	No one reads my blog.	[start] Personne ne lit mon journal électroniq...
105652	This can't wait until tomorrow.	[start] Ceci ne peut pas attendre jusqu'à dema...
149809	I never thought I'd find a woman like you.	[start] Je n'ai jamais pensé que je trouverais...
31207	That doesn't add up.	[start] Le compte n'y est pas. [end]
84658	My customers never complain.	[start] Mes clients ne se plaignent jamais. [end]

In [5]:
```python
#shuffle the data
df = df.sample(frac=1).reset_index(drop=True)

# split the data into train, validation, and test sets
train_size = int(len(df) * 0.7)
val_size = int(len(df) * 0.2)
test_size = int(len(df) * 0.1)

train_df = df[:train_size]
val_df = df[train_size:train_size+val_size]
test_df = df[train_size+val_size:]

# display the data sets representations using a pie chart just to see the distribution of the data
labels = 'Train', 'Validation', 'Test'
sizes = [len(train_df), len(val_df), len(test_df)]
explode = (0.1, 0, 0)
fig1, ax1 = plt.subplots()
ax1.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%', shadow=True, startangle=90)
ax1.axis('equal')
plt.show()
```



In [6]:
```python
max_tokens = 25000
sequence_length = 30

# define a custom standardization function that convert to lowercase and strips all punctuations except "[" and "]" (so we can tell apart "start" from "[start]").
strip_chars = string.punctuation
strip_chars = strip_chars.replace("[", "")
strip_chars = strip_chars.replace("]", "")

def custom_standardization(input_string):
    lowercase = tf.strings.lower(input_string)
    return tf.strings.regex_replace(
        lowercase, f"[{re.escape(strip_chars)}]", "")

# tokenize the data using our custom standardization function
source_vectorization = keras.layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=sequence_length,
)
target_vectorization = keras.layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=sequence_length + 1, # add +1 token to our target sentences since they'll be shifted right by 1 during training
    standardize=custom_standardization,
)

# index all tokens in the source and target sentences
train_source_texts = train_df['source'].values
train_target_texts = train_df['target'].values
source_vectorization.adapt(train_source_texts)
target_vectorization.adapt(train_target_texts)
```

```
I0000 00:00:1764580060.938560      47 gpu_device.cc:2022] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 13942 MB memory:  -> device: 0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1764580060.939257      47 gpu_device.cc:2022] Created device /job:localhost/replica:0/task:0/device:GPU:1 with 13942 MB memory:  -> device: 1, name: Tesla T4, pci bus id: 0000:00:05.0, compute capability: 7.5
```

In [7]:
```python
# display a random sample before and after vectorization just to test the vectorization
random_sample = random.randint(0, len(train_df))
print("Source texts (one random sample):", train_source_texts[random_sample])
print("Target texts (one random sample):", train_target_texts[random_sample])
print("Source vectors (one random sample):", source_vectorization(train_source_texts[random_sample]))
print("Target vectors (one random sample):", target_vectorization(train_target_texts[random_sample]))

# display the decoding of the vectorized text (from vector back to text) just to test the vectorization
source_decoded_text = ''
for i in range(len(source_vectorization(train_source_texts[random_sample]))):
    source_decoded_text += source_vectorization.get_vocabulary()[source_vectorization(train_source_texts[random_sample])[i]] + ' '
print("Source decoded texts (one random sample):", source_decoded_text)

target_decoded_text = ''
for i in range(len(target_vectorization(train_target_texts[random_sample]))):
    target_decoded_text += target_vectorization.get_vocabulary()[target_vectorization(train_target_texts[random_sample])[i]] + ' '
print("Target decoded texts (one random sample):", target_decoded_text)
```

```
Source texts (one random sample): I want to travel by airplane.
Target texts (one random sample): [start] Je veux voyager en avion. [end]
Source vectors (one random sample): tf.Tensor(
[   2   28    4  926   88 1863    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0], shape=(30,), dtype=int64)
Target vectors (one random sample): tf.Tensor(
[   2    4   43  894   21 1380    3    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0], shape=(31,), dtype=int64)
Source decoded texts (one random sample): i want to travel by airplane
Target decoded texts (one random sample): [start] je veux voyager en avion [end]
```

In [8]:
```python
# display the shape of our vectorized data
train_source_vectors = source_vectorization(train_source_texts)
train_target_vectors = target_vectorization(train_target_texts)
print("Source vectors (shape):", train_source_vectors.shape)
print("Target vectors (shape):", train_target_vectors.shape)
```

```
Source vectors (shape): (122934, 30)
Target vectors (shape): (122934, 31)
```

In [9]:
```python
class PositionalEmbedding(keras.layers.Layer):
    def __init__(self, sequence_length, input_dim, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.token_embeddings = keras.layers.Embedding(input_dim=input_dim, output_dim=output_dim) # token embedding layer
        self.position_embeddings = keras.layers.Embedding(input_dim=sequence_length, output_dim=output_dim) # position embedding layer
        self.sequence_length = sequence_length
        self.input_dim = input_dim
        self.output_dim = output_dim

    def call(self, inputs):
        embedded_tokens = self.token_embeddings(inputs) # embed the tokens
        length = tf.shape(inputs)[-1]
        positions = tf.range(start=0, limit=length, delta=1) # create the positional information
        embedded_positions = self.position_embeddings(positions) # embed the positions
        return embedded_tokens + embedded_positions # add the token and position embeddings to create the positional embeddings

    def compute_mask(self, inputs, mask=None):
        return keras.ops.not_equal(inputs, 0)

    def get_config(self):
        config = super(PositionalEmbedding, self).get_config()
        config.update({
            "input_dim": self.input_dim,
            "output_dim": self.output_dim,
            "sequence_length": self.sequence_length,
        })
        return config
```

In [10]:
```python
# display a random sample before and after embbeding just to test our class

embed_dim = 256

with tf.device('cpu:0'):
    train_source_embedded = PositionalEmbedding(
        sequence_length=sequence_length,
        input_dim=max_tokens,
        output_dim=embed_dim,
        name="source_embedding",
    ) (train_source_vectors)

    train_target_embedded = PositionalEmbedding(
        sequence_length=sequence_length,
        input_dim=max_tokens,
        output_dim=embed_dim,
        name="target_embedding",
    ) (train_source_vectors)

    random_sample = random.randint(0, len(train_df))
    print("Source texts (one random sample):", train_source_texts[random_sample])
    print("Target texts (one random sample):", train_target_texts[random_sample])
    print("Source vectors (one random sample):", source_vectorization(train_source_texts[random_sample]))
    print("Target vectors (one random sample):", target_vectorization(train_target_texts[random_sample]))
    print("Source embedded vectors (one random sample):", train_source_embedded[random_sample])
    print("Target embedded vectors (one random sample):", train_target_embedded[random_sample])
```

```
Source texts (one random sample): He plays golf every Sunday.
Target texts (one random sample): [start] Il joue au golf tous les dimanches. [end]
Source vectors (one random sample): tf.Tensor(
[  10 1024 1421  186  851    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0], shape=(30,), dtype=int64)
Target vectors (one random sample): tf.Tensor(
[   2   13  534   45 1774   81   22 6099    3    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0], shape=(31,), dtype=int64)
Source embedded vectors (one random sample): tf.Tensor(
[[ 0.026296    0.0216063   0.0688701  ...  0.01396178  0.00625681
   0.01317896]
 [ 0.02303186 -0.01465917  0.02694832 ... -0.03492904 -0.04391035
  -0.04686096]
 [ 0.03277893 -0.04117743 -0.05859209 ... -0.03558567  0.01869255
   0.01502343]
 ...
 [ 0.06389515 -0.0323446  -0.00807246 ...  0.0046027   0.00812168
  -0.05086654]
 [ 0.06651908 -0.01516302  0.01140884 ...  0.00702825  0.00242268
  -0.05428128]
 [ 0.03795744 -0.03111267 -0.00141303 ... -0.0034618   0.01209534
  -0.07160113]], shape=(30, 256), dtype=float32)
Target embedded vectors (one random sample): tf.Tensor(
[[ 0.02309406  0.06105688  0.06795082 ... -0.03965486  0.01948507
   0.00692154]
 [ 0.00848028 -0.039036    0.04537767 ... -0.06131773  0.00664019
  -0.01418153]
 [-0.08057331 -0.07200775 -0.01640376 ...  0.06788531 -0.01977504
   0.04724215]
 ...
 [ 0.05153641  0.00979468 -0.0593577  ...  0.00319338 -0.07438785
  -0.04874445]
 [ 0.07438993  0.00385368 -0.09514556 ... -0.05619801  0.01331915
   0.0099447 ]
 [-0.00812847 -0.05105235 -0.04634652 ... -0.03703866 -0.05124195
  -0.08639123]], shape=(30, 256), dtype=float32)
```

In [11]:
```python
# display the shape of our embedded data just to test the class
print("Source embedded vectors (shape):", train_source_embedded.shape)
print("Target embedded vectors (shape):", train_target_embedded.shape)
```

```
Source embedded vectors (shape): (122934, 30, 256)
Target embedded vectors (shape): (122934, 30, 256)
```

In [12]:
```python
# credits to OpenAI for that one (https://github.com/openai/gpt-2/blob/master/src/model.py)

def shape_list(x):
    """Deal with dynamic shape in tensorflow cleanly."""
    static = x.shape.as_list()
    dynamic = tf.shape(x)
    return [dynamic[i] if s is None else s for i, s in enumerate(static)]

def attention_mask(nd, ns, *, dtype):
    """1's in the lower triangle, counting from the lower right corner.
    Same as tf.matrix_band_part(tf.ones([nd, ns]), -1, ns-nd), but doesn't produce garbage on TPUs.
    """
    i = tf.range(nd)[:,None]
    j = tf.range(ns)
    m = i >= j - ns + nd
    return tf.cast(m, dtype)

def mask_attn_weights(w):
    # w has shape [batch, heads, dst_sequence, src_sequence], where information flows from src to dst.
    _, _, nd, ns = shape_list(w)
    b = attention_mask(nd, ns, dtype=w.dtype)
    b = tf.reshape(b, [1, 1, nd, ns])
    w = w*b - tf.cast(1e10, w.dtype)*(1-b)
    return w
```

In [13]:
```python
# display the causal masking of a random tensor just to test the function
random_tensor = tf.random.uniform(shape=(1, 1, 5, 5), minval=0, maxval=1, dtype=tf.float32)
print("Masked attention weights:", mask_attn_weights(random_tensor))
```

```
Masked attention weights: tf.Tensor(
[[[[ 2.9519212e-01 -1.0000000e+10 -1.0000000e+10 -1.0000000e+10
    -1.0000000e+10]
   [ 8.5570455e-01  5.0791478e-01 -1.0000000e+10 -1.0000000e+10
    -1.0000000e+10]
   [ 3.6504662e-01  1.5048063e-01  8.4397960e-01 -1.0000000e+10
    -1.0000000e+10]
   [ 6.9107974e-01  2.0170975e-01  8.6676407e-01  6.7780781e-01
    -1.0000000e+10]
   [ 8.4972739e-01  3.9732981e-01  5.4674041e-01  4.9651253e-01
     7.9733622e-01]]]], shape=(1, 1, 5, 5), dtype=float32)
```

In [14]:
```python
def scaled_dot_product_attention(q, k, v, use_causal_mask=False):
    d_k = tf.cast(tf.shape(k)[-1], tf.float32)
    scores = tf.matmul(q, k, transpose_b=True) # Matmul of Q and K
    scaled_scores = scores / tf.math.sqrt(d_k) # Scale
    if use_causal_mask:
        scaled_scores = mask_attn_weights(scaled_scores) # Mask (opt.)
    weights = tf.nn.softmax(scaled_scores, axis=-1) # SoftMax
    output = tf.matmul(weights, v) # Matmul of SoftMax and V
    return output
```

In [15]:
```python
# display the shape of our attention output just to test the function
with tf.device('cpu:0'):
    input = train_source_embedded
    input = tf.expand_dims(input, axis=1)
    print("Scaled dot product attention (shape):", scaled_dot_product_attention(input, input, input, use_causal_mask=True).shape)
```

```
Scaled dot product attention (shape): (122934, 1, 30, 256)
```

In [16]:
```python
class MultiHeadAttention(keras.layers.Layer):
    def __init__(self, embed_dim, h, **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.h = h
        if embed_dim % h != 0:
            raise ValueError(
                f"dimension of the embedding space = {embed_dim} should be divisible by number of heads = {h}"
            )
        self.q_linear = keras.layers.Dense(embed_dim)
        self.k_linear = keras.layers.Dense(embed_dim)
        self.v_linear = keras.layers.Dense(embed_dim)
        self.concat_linear = keras.layers.Dense(embed_dim)

    def split_heads(self, x, batch_size):
        x = tf.reshape(x, shape=(batch_size, -1, self.h, self.embed_dim // self.h))
        return tf.transpose(x, perm=[0, 2, 1, 3])

    def concat_heads(self, x, batch_size):
        x = tf.transpose(x, perm=[0, 2, 1, 3])
        return tf.reshape(x, (batch_size, -1, self.embed_dim))

    def call(self, q, k, v, use_causal_mask=False):
        batch_size = tf.shape(k)[0]
        q = self.q_linear(q)
        k = self.k_linear(k)
        v = self.v_linear(v)
        q = self.split_heads(q, batch_size)
        k = self.split_heads(k, batch_size)
        v = self.split_heads(v, batch_size)
        attention = scaled_dot_product_attention(q, k, v, use_causal_mask)
        concat = self.concat_heads(attention, batch_size)
        concat = self.concat_linear(concat)
        return concat

    def get_config(self):
        config = super(MultiHeadAttention, self).get_config()
        config.update({
            "embed_dim": self.embed_dim,
            "h": self.h,
        })
        return config
```

In [17]:
```python
class TransformerEncoder(keras.layers.Layer):
    def __init__(self, embed_dim, dense_dim, num_heads, **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.dense_dim = dense_dim
        self.num_heads = num_heads
        self.layer_norm_1 = keras.layers.LayerNormalization()
        self.layer_norm_2 = keras.layers.LayerNormalization()
        self.global_self_attention = MultiHeadAttention(embed_dim=embed_dim, h=num_heads)
        self.feed_forward = keras.Sequential(
            [keras.layers.Dense(dense_dim, activation="relu"),
             keras.layers.Dense(embed_dim),]
        )

    def call(self, x):
        # Post layer normalization + residual connections
        x = self.layer_norm_1(x + self.global_self_attention(q=x, k=x, v=x))
        x = self.layer_norm_2(x + self.feed_forward(x))
        return x

    def get_config(self):
        config = super().get_config()
        config.update({
            "embed_dim": self.embed_dim,
            "dense_dim": self.dense_dim,
            "num_heads": self.num_heads,
        })
        return config
```

In [18]:
```python
class TransformerDecoder(keras.layers.Layer):
    def __init__(self, embed_dim, dense_dim, num_heads, **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.dense_dim = dense_dim
        self.num_heads = num_heads
        self.causal_self_attention = MultiHeadAttention(embed_dim=embed_dim, h=num_heads)
        self.cross_attention = MultiHeadAttention(embed_dim=embed_dim, h=num_heads)
        self.feed_forward = keras.Sequential(
            [keras.layers.Dense(dense_dim, activation="relu"),
             keras.layers.Dense(embed_dim),]
        )
        self.layer_norm_1 = keras.layers.LayerNormalization()
        self.layer_norm_2 = keras.layers.LayerNormalization()
        self.layer_norm_3 = keras.layers.LayerNormalization()

    def get_config(self):
        config = super().get_config()
        config.update({
            "embed_dim": self.embed_dim,
            "dense_dim": self.dense_dim,
            "num_heads": self.num_heads,
        })
        return config

    def call(self, x, context):
        # Post layer normalization + residual connections
        x = self.layer_norm_1(x + self.causal_self_attention(q=x, k=x, v=x, use_causal_mask=True))
        x = self.layer_norm_2(x + self.cross_attention(q=x, k=context, v=context))
        x = self.layer_norm_3(x + self.feed_forward(x))
        return x
```

In [19]:
```python
batch_size = 64

def format_dataset(source, target):
    source_vectors = source_vectorization(source)
    target_vectors = target_vectorization(target)
    return ({
        "source": source_vectors, # encoder_inputs
        "target": target_vectors[:, :-1], # decoder_inputs (truncate by 1 to keep it at the same length as decoder_outputs, which is shifted right by 1).
    }, target_vectors[:, 1:]) # decoder_outputs

def make_dataset(df):
    dataset = tf.data.Dataset.from_tensor_slices((df["source"].values, df["target"].values))
    dataset = dataset.batch(batch_size)
    dataset = dataset.map(format_dataset, num_parallel_calls=4)
    return dataset.shuffle(2048).prefetch(16).cache()

train_ds = make_dataset(train_df)
val_ds = make_dataset(val_df)
```

In [20]:
```python
# display the shape of the first batch of data in the dataset just to see what it looks like
for batch in train_ds.take(1):
    print("Encoder Inputs:", batch[0]["source"].shape)
    print("Decoder Inputs:", batch[0]["target"].shape)
    print("Decoder Outputs:", batch[1].shape)
```

```
Encoder Inputs: (64, 30)
Decoder Inputs: (64, 30)
Decoder Outputs: (64, 30)
```

In [21]:
```python
embed_dim = 512 # dimension of the embedding space
dense_dim = 2048 # dimension of the feed forward network (a rule of thumb is to use 4 times the size of the embeddings)
num_heads = 8

# the transformer body
encoder_inputs = keras.Input(shape=(None,), dtype="int64", name="source")
x = PositionalEmbedding(sequence_length, max_tokens, embed_dim)(encoder_inputs)
encoder_outputs = TransformerEncoder(embed_dim, dense_dim, num_heads)(x)
decoder_inputs = keras.Input(shape=(None,), dtype="int64", name="target")
x = PositionalEmbedding(sequence_length, max_tokens, embed_dim)(decoder_inputs)
x = TransformerDecoder(embed_dim, dense_dim, num_heads)(x, encoder_outputs)

# the transformer head
x = keras.layers.Dropout(0.5)(x)
decoder_outputs = keras.layers.Dense(max_tokens, activation="softmax")(x)

transformer = keras.Model([encoder_inputs, decoder_inputs], decoder_outputs)
```

Training the Transformer

In [22]:
```python
transformer.compile(
    optimizer="rmsprop",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])

EPOCHS = 50
checkpoint_filepath = '/tmp/checkpoint.weights.h5'
callbacks_list = [
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.1,
        patience=3,
    ),
    keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=6,
    ),
    keras.callbacks.ModelCheckpoint(
        filepath=checkpoint_filepath,
        save_weights_only=True,
        monitor='val_loss',
        mode='min',
        save_best_only=True
    ),
]

transformer.fit(train_ds,
                epochs=EPOCHS,
                callbacks=callbacks_list,
                validation_data=val_ds)

transformer.load_weights(checkpoint_filepath)
```

```
Epoch 1/50
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1764580117.117646     109 service.cc:148] XLA service 0x786ecc0903a0 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:
I0000 00:00:1764580117.118589     109 service.cc:156]   StreamExecutor device (0): Tesla T4, Compute Capability 7.5
I0000 00:00:1764580117.118608     109 service.cc:156]   StreamExecutor device (1): Tesla T4, Compute Capability 7.5
W0000 00:00:1764580117.421515     109 assert_op.cc:38] Ignoring Assert operator compile_loss/sparse_categorical_crossentropy/SparseSoftmaxCrossEntropyWithLogits/assert_equal_1/Assert/Assert
I0000 00:00:1764580118.538670     109 cuda_dnn.cc:529] Loaded cuDNN version 90300
   1/1921 ━━━━━━━━━━━━━━━━━━━━ 10:47:32 20s/step - accuracy: 0.0000e+00 - loss: 10.3950
I0000 00:00:1764580131.026786     109 device_compiler.h:188] Compiled cluster using XLA!  This line is logged at most once for the lifetime of the process.
 966/1921 ━━━━━━━━━━━━━━━━━━━━ 2:09 135ms/step - accuracy: 0.7895 - loss: 1.6628
W0000 00:00:1764580262.114321     110 assert_op.cc:38] Ignoring Assert operator compile_loss/sparse_categorical_crossentropy/SparseSoftmaxCrossEntropyWithLogits/assert_equal_1/Assert/Assert
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 0s 150ms/step - accuracy: 0.7982 - loss: 1.4764
W0000 00:00:1764580420.084812     110 assert_op.cc:38] Ignoring Assert operator compile_loss/sparse_categorical_crossentropy/SparseSoftmaxCrossEntropyWithLogits/assert_equal_1/Assert/Assert
W0000 00:00:1764580438.949424     109 assert_op.cc:38] Ignoring Assert operator compile_loss/sparse_categorical_crossentropy/SparseSoftmaxCrossEntropyWithLogits/assert_equal_1/Assert/Assert
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 332s 162ms/step - accuracy: 0.7982 - loss: 1.4763 - val_accuracy: 0.8198 - val_loss: 1.0686 - learning_rate: 0.0010
Epoch 2/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 312s 163ms/step - accuracy: 0.8511 - loss: 0.9094 - val_accuracy: 0.9021 - val_loss: 0.5620 - learning_rate: 0.0010
Epoch 3/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 311s 162ms/step - accuracy: 0.9016 - loss: 0.5696 - val_accuracy: 0.9126 - val_loss: 0.4838 - learning_rate: 0.0010
Epoch 4/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 311s 162ms/step - accuracy: 0.9140 - loss: 0.4708 - val_accuracy: 0.9183 - val_loss: 0.4486 - learning_rate: 0.0010
Epoch 5/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 312s 163ms/step - accuracy: 0.9216 - loss: 0.4122 - val_accuracy: 0.9204 - val_loss: 0.4373 - learning_rate: 0.0010
Epoch 6/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 312s 162ms/step - accuracy: 0.9276 - loss: 0.3685 - val_accuracy: 0.9228 - val_loss: 0.4278 - learning_rate: 0.0010
Epoch 7/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 311s 162ms/step - accuracy: 0.9326 - loss: 0.3359 - val_accuracy: 0.9237 - val_loss: 0.4255 - learning_rate: 0.0010
Epoch 8/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 310s 161ms/step - accuracy: 0.9367 - loss: 0.3068 - val_accuracy: 0.9254 - val_loss: 0.4261 - learning_rate: 0.0010
Epoch 9/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 311s 162ms/step - accuracy: 0.9406 - loss: 0.2822 - val_accuracy: 0.9263 - val_loss: 0.4254 - learning_rate: 0.0010
Epoch 10/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 312s 162ms/step - accuracy: 0.9441 - loss: 0.2596 - val_accuracy: 0.9267 - val_loss: 0.4230 - learning_rate: 0.0010
Epoch 11/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 310s 161ms/step - accuracy: 0.9471 - loss: 0.2416 - val_accuracy: 0.9272 - val_loss: 0.4263 - learning_rate: 0.0010
Epoch 12/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 309s 161ms/step - accuracy: 0.9497 - loss: 0.2255 - val_accuracy: 0.9280 - val_loss: 0.4276 - learning_rate: 0.0010
Epoch 13/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 310s 161ms/step - accuracy: 0.9523 - loss: 0.2109 - val_accuracy: 0.9279 - val_loss: 0.4308 - learning_rate: 0.0010
Epoch 14/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 311s 162ms/step - accuracy: 0.9594 - loss: 0.1701 - val_accuracy: 0.9355 - val_loss: 0.3767 - learning_rate: 1.0000e-04
Epoch 15/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 311s 162ms/step - accuracy: 0.9669 - loss: 0.1304 - val_accuracy: 0.9360 - val_loss: 0.3733 - learning_rate: 1.0000e-04
Epoch 16/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 310s 161ms/step - accuracy: 0.9696 - loss: 0.1169 - val_accuracy: 0.9361 - val_loss: 0.3737 - learning_rate: 1.0000e-04
Epoch 17/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 310s 161ms/step - accuracy: 0.9715 - loss: 0.1076 - val_accuracy: 0.9363 - val_loss: 0.3748 - learning_rate: 1.0000e-04
Epoch 18/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 309s 161ms/step - accuracy: 0.9730 - loss: 0.1010 - val_accuracy: 0.9363 - val_loss: 0.3758 - learning_rate: 1.0000e-04
Epoch 19/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 311s 162ms/step - accuracy: 0.9740 - loss: 0.0957 - val_accuracy: 0.9365 - val_loss: 0.3746 - learning_rate: 1.0000e-05
Epoch 20/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 310s 162ms/step - accuracy: 0.9746 - loss: 0.0933 - val_accuracy: 0.9365 - val_loss: 0.3744 - learning_rate: 1.0000e-05
Epoch 21/50
1921/1921 ━━━━━━━━━━━━━━━━━━━━ 309s 161ms/step - accuracy: 0.9748 - loss: 0.0922 - val_accuracy: 0.9365 - val_loss: 0.3745 - learning_rate: 1.0000e-05
```

Testing the Transformer

In [23]:
```python
target_vocab = target_vectorization.get_vocabulary()
target_index_lookup = dict(zip(range(len(target_vocab)), target_vocab))
max_decoded_sentence_length = 30

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization(
            [decoded_sentence])[:, :-1]
        predictions = transformer(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(predictions[0, i, :])
        sampled_token = target_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

# let's translate 50 random sentences
for i in range(50):
    random_index = np.random.randint(0, len(test_df))
    input_sentence = test_df["source"].iloc[random_index]
    print(input_sentence)
    print(decode_sequence(input_sentence))
    print()
```

```
The door will not open.
[start] la porte refuse de souvrir [end]

I didn't see who was driving.
[start] je nai pas vu qui était en train de conduire [end]

I can confirm this.
[start] je peux confirmer ceci [end]

I taught my wife how to drive.
[start] jai appris à conduire ma femme [end]

I'm the oldest.
[start] je suis le plus vieux [end]

We're just talking.
[start] nous ne parlons que de nous [end]

Do you travel often?
[start] voyagezvous souvent [end]

She made her mother happy.
[start] elle a rendu sa mère heureuse [end]

The car was stuck in the mud.
[start] la voiture était immobilisée dans la boue [end]

I thought you wanted the truth.
[start] je pensais que tu voulais la vérité [end]

You're not a very good liar.
[start] vous nêtes pas un très bon menteur [end]

Was it really necessary to do that?
[start] Êtaitce vraiment nécessaire de faire ça [end]

Some children are swimming in the sea.
[start] quelques enfants se baignent dans la mer [end]

Are you sure it was Tom?
[start] Êtesvous sûres que cétait tom [end]

Maybe there's a better way to do this.
[start] peutêtre il y a une meilleure façon de faire ceci [end]

Do you want to talk about anything?
[start] voulezvous parler de quoi que ce soit [end]

He beckoned me to follow him.
[start] il me fit signe de suivre [end]

The house is opposite the church. You can't miss it.
[start] la maison est la plus lopposé que tu ne peux pas le manquer [end]

I used to use Twitter, but then found it a bit boring, so I stopped using it.
[start] je me suis habitué à utiliser un peu alors [UNK] [end]

I never meant to hurt any of you.
[start] je nai jamais voulu vous faire de mal [end]

Our cat is missing.
[start] notre chat disparu [end]

She has a flower in her hand.
[start] elle a une fleur dans la main [end]

I found out whose car went off the road.
[start] jai découvert à qui est allé de la voiture de la route [end]

What an interesting party!
[start] quelle fête est une fête intéressante [end]

I can wait for you.
[start] je peux tattendre [end]

Who is it?
[start] qui estce [end]

That guy is an outlaw.
[start] ce type est un type [UNK] [end]

I have lived here a long time.
[start] jai vécu ici longtemps [end]

Do you think I'm being unreasonable?
[start] pensezvous que je sois déraisonnable [end]

I can't even lift this small box.
[start] je ne peux même pas soulever cette boîte [end]

Life is short.
[start] la vie est petite [end]

It's something they always wanted to do.
[start] cest quelque chose quelles voulaient toujours faire [end]

We enjoyed watching TV.
[start] nous avons apprécié regarder la télé [end]

They tried to discourage him from going.
[start] ils essayèrent de le décourager dy aller [end]

That can't be true.
[start] Ça ne peut être vrai [end]

I have no children.
[start] je nai pas denfants [end]

This is my beer.
[start] cest ma bière [end]

You're very attractive.
[start] vous êtes fort attirants [end]

How are you now?
[start] comment ça se passe maintenant [end]

My grandpa is out of step with the younger generation.
[start] mon papy est en dehors de moins de moins de moins jeune [end]

It is stupid of you to believe in him.
[start] cest stupide de ta part de le croire en vérité [end]

He noticed straight away.
[start] il a remarqué directement [end]

I want my life back.
[start] je veux que tu me rendes ma vie [end]

He stuck to his promise.
[start] il a une promesse à sa promesse [end]

This chair is very comfortable.
[start] cette chaise est très confortable [end]

Bring the key.
[start] apporte la clé [end]

Let's finish the job.
[start] jouons au travail [end]

This time, you've gone too far.
[start] cette fois tu es allée trop loin [end]

I'm too tired to concentrate on this problem right now.
[start] je suis trop fatigué pour me concentrer sur ce problème maintenant [end]

They're going to make mistakes.
```

```
[start] ils vont faire des erreurs [end]
```

```
In [ ]:
```